5. Action Tasks

Walk through the example, try small changes, and run short experiments:

Q1: What happens if you change the PGD parameters (steps, number of random restarts)?

- **Steps** = how many times PGD tries to move the image slightly.
- **Restarts** = how many random starting points PGD tries.

The paper "Grokking" (https://arxiv.org/pdf/2201.02177) suggests that giving the model more training time so it stops memorising data and starts learning the true rule. PGD steps/restarts, however, mean stronger adversarial attacks during training, they test how well the model keeps the rule under different input changes. They both aim for deeper understanding, but they work in different ways.

Effect of PGD Steps and Restarts on Property-Driven Training (ε = 8/255):

Steps	Restarts	Train Prediction Accuracy	Train Constraint Accuracy	Train Constraint Security	Test Prediction Accuracy	Test Constraint Accuracy	Test Constraint Security	Notes
2	1	0.9718	0.9926	0.9963	0.8750	1.0000	0.0000	Very fast; not robust to attacks
5	5	0.9933	1.0000	0.9963	0.8333	1.0000	0.1250	Medium speed; small security improvement
20	10	0.9804	0.9963	0.9926	0.7500	1.0000	0.8125	Slower; best balance of accuracy and robustness
70	40	0.9902	1.0000	0.9890	0.7812	1.0000	0.2500	Very slow (~15 min); overfitting or unstable

When we changed the PGD parameters, we saw how the strength of the attack during training affected how well the model learned the logical rule : *“do not predict two opposite faces at the same time.”*

- With **low attack values (2 steps, 1 restart)**: training was very fast, and accuracy was high, but the model was not truly robust as it easily broke the rule when attacked.
- With **medium attack values (5 steps, 5 restarts)**: the model became slightly better at following the rule, but robustness was still weak.
- With **higher attack values (20 steps, 10 restarts)**: training took longer, but the model learned the rule more strongly, keeping good performance even when attacked. This setting gave the best overall balance between accuracy and robustness.
- With **very high values (70 steps, 40 restarts)**: training became very slow and the model’s robustness dropped again, likely because the attack was too strong and confused the learning process.

Overall, when we increase the number of PGD steps and restarts, the model learns the rule more carefully and behaves in a more logical way , a bit like in Grokking, where the model moves from just memorising to truly understanding, but only up to a point. Because even if the model seems to follow the rule under stronger PGD, this is not a full guarantee, we still need formal verification to provide provable assurance rather than empirical robustness.

Q2: The network might learn a shortcut and vacuously satisfy the constraint "do not predict two opposite faces at the same time" by predicting no faces at all. Can you make changes to the constraint that to this behaviour?

To fix that, I would add another logical rule that checks if the sum of positive outputs (predicted faces) is at least 1.
This means the network must always predict at least one face, so it can’t satisfy the “no opposite faces” rule by predicting none.

In code:

```python
def get_postcondition(self, N: torch.nn.Module, x_adv: torch.Tensor) -> Callable[[Logic], torch.Tensor]:
    y_adv = N(x_adv)
    num_faces = y_adv.shape[1]  # should be 6

    # Combine both constraints inside the logic lambda:
    # 1) No opposite faces together
    # 2) At least one face predicted (some logit > 0)
    return lambda logic: logic.AND(
        # (1) no-opposites
        logic.AND(
        *[
            logic.OR(
                logic.LEQ(y_adv[:, i], self.zero),  # either label i is not predicted ...
                logic.LEQ(y_adv[:, j], self.zero),  # ... or label j is not predicted
            )
            for i, j in self.opposingFacePairs
        ]
    ),
        # (2) at-least-one (some logit > 0)
        logic.OR(
            *[logic.GT(y_adv[:, k], self.zero) for k in range(num_faces)]
        )
    )
```

After adding the **at-least-one-face** constraint (**5 steps, 5 restarts**):

- **Prediction Accuracy (0.69):** dropped a bit compared to before (0.83) because the model now has less “easy shortcuts.” It must always predict at least one face, so it can’t satisfy the rule by staying silent.
- **Constraint Accuracy (1.0)**: shows the model fully respected the logical rule on normal (non-attacked) images, it never predicted impossible or empty face combinations.
- **Constraint Security (0.875):** means even when attacked by PGD, it still followed the rule in **87.5%** of cases, which is much stronger than before where the model could satisfy constraints slightly by predicting nothing.

Overall, the new rule forced the network to make meaningful predictions while still following the logical constraints. Training produced a model that behaves more robustly.

Q3: Can you come up with any other meaningful constraints, such as a modified version of the epsilon-delta robustness one we have seen in the lectures, or something like "always predict exactly three faces"?

The **ε-δ robustness** : small changes in the input should not cause big changes in the output.

We could add a robustness constraint for the dice example that works like this:

- For a small change in the image like lighting or position of the dice image (within an ε-ball), the output predictions (the face logits) shouldn’t change too much.

- This constraint ensures **consistent outputs** under small input perturbations, improving stability.
- It could be combined with the logical dice rules (no opposite faces, at least one face) for even more reliable predictions.

However, there can be **downsides** to adding too many logical constraints during training. Even though they can improve accuracy and consistency, they can also:

- Make the model harder to understand because multiple rules might conflict.
- Reduce generalisation, meaning the model becomes too specialised for one task and may not perform well on new data.
- Slow down training or make it harder for the model to find the best balance between all rules.

Q4: Lastly, think about how you might apply the library in your own work.

I'd apply it to *Maah* Companion Robot:

1. Logical Rule for Safe Emotional Response

I could express safe emotional reaction as logic constraints:

"If the user's touch is gentle and long, Maah's vibration intensity must not exceed a comfort threshold."

Formally, if: **p** = touch pressure, **d** = touch duration, **v** = vibration intensity, and **Tp**, **Td**, **Tv** are their safe threshold limits, then the rule could be written as:

$$(p < Tp \land d > Td) \Rightarrow v \leq Tv$$

This ensures that gentle, comforting touches always produce soft and safe feedback.

2. Preventing Contradictory Behaviours

"Maah should not show both *comfort* and *alert* behaviours at the same time."

$$\neg(\text{comfort}(y) \land \text{alert}(y))$$

In practice, this could be implemented as a *loss contradiction* function that penalises the network if both outputs are activated together beyond a small limit.

3. Robustness to Small Sensor Noise (ε–δ Robustness)

"Small changes in input (touch readings) should not cause large changes in output (vibration or emotion)"

If the tactile sensors change slightly due to noise or user movement, Maah's response should stay consistent:

$$|| x - x' ||< \epsilon \Rightarrow || f(x) - f(x') ||< \delta$$

It improves both stability and user trust, especially during long-term interactions.